

CS247 – SPRING 26 – TUTORIAL 2

M3 – VALUE VS. ENTITY OBJECTS, INFORMATION HIDING

M4 – SPECIAL MEMBER FUNCTIONS



1. Entity Objects – Things to Remember

- **Identity is by address, not value** — two Passenger objects with identical fields are still two different people
- **Delete the copy constructor and `operator=`** explicitly with `= delete`; do not rely on `private:` to hide them
- **Avoid `operator==`** unless you redefine it to mean "same identity" (e.g., compare unique IDs, not field-by-field)
- **Hold entities by pointer or reference**, never by value — passing one by value should not even compile
- `operator<` can be useful if (and only if) you anchor it to a unique key like an ID, never to mutable state



1. Entity Objects – Example

```
class Airplane {
private:
    std::string tailNumber_; // unique real-world identifier
    int fuelLevel_;
public:
    explicit Airplane(std::string tail) : tailNumber_{std::move(tail)}, fuelLevel_{0} {}

    // Entities cannot be duplicated => there is only one AC-001 in the world
    Airplane(const Airplane&) = delete;
    Airplane& operator=(const Airplane&) = delete;

    // In this specific Airplane design, we also strictly prohibit moving
    Airplane(Airplane&&) = delete;
    Airplane& operator=(Airplane&&) = delete;

    // Equality compares identity, not state => same tail = same plane
    friend bool operator==(const Airplane& a, const Airplane& b) {
        return a.tailNumber_ == b.tailNumber_;
    }

    void refuel(int liters) { fuelLevel_ += liters; } // mutator: entities change
};
```

1. Entity Objects – Challenge

Will this compile, and if so, what does it print?

```
class Passenger {  
public:  
    std::string name;  
    Passenger(std::string n) : name{std::move(n)} {}  
    Passenger(const Passenger&) = delete;  
};  
  
void announce(Passenger p) { std::cout << p.name << '\n'; }  
  
int main() {  
    Passenger alice{"Alice"};  
    announce(std::move(alice)); // What happens?  
}
```

1. Entity Objects – Challenge (Solution)

Will this compile, and if so, what does it print?

```
class Passenger {  
public:  
    std::string name;  
    Passenger(std::string n) : name{std::move(n)} {}  
    Passenger(const Passenger&) = delete;  
};  
  
void announce(Passenger p) { std::cout << p.name << '\n'; }  
  
int main() {  
    Passenger alice{"Alice"};  
    announce(std::move(alice)); // What happens?  
}
```

Answer: This fails to compile.

Deleting the copy constructor also prevents the compiler from generating a move constructor (*see the C++17 special-members table*).

`announce` takes its parameter by value, requiring either a copy or move, both unavailable.

Fix: explicitly declare the move constructor or pass by `const Passenger&`.

1. Entity Objects – Challenge (Solution)

Will this compile, and if so, what does it print?

```
class Passenger {  
public:  
    std::string name;  
    Passenger(std::string n) : name{std::move(n)} {}  
    Passenger(const Passenger&) = delete;  
};  
  
void announce(const Passenger& p) { std::cout << p.name << '\n'; }  
  
int main() {  
    Passenger alice{"Alice"};  
    announce(std::move(alice));  
}
```

Answer: This fails to compile.

Deleting the copy constructor also prevents the compiler from generating a move constructor (*see the C++17 special-members table*).

`announce` takes its parameter by value, requiring either a copy or move, both unavailable.

Fix: explicitly declare the move constructor or pass by `const Passenger&`.

2. Value Objects – Things to Remember

- **Define `operator==`** based on field equality — two `Rational(1,2)` objects must compare equal regardless of address
- **Provide deep copy semantics** — copying a value must yield an independent instance with no shared resources
- **Overload arithmetic where it makes domain sense**, but resist overloading operators whose meaning would surprise a reader
- **Avoid virtual functions** in value types — polymorphism on values leads to object slicing and conceptual confusion
- **Prefer pass-by-const reference for inputs** to avoid unnecessary copies, even though values are copyable

2. Value Objects – Example

```
class Rational {  
private:  
    int num_, denom_;  
public:  
    explicit Rational(int n = 0, int d = 1) : num_{n}, denom_{d} {  
        if (d == 0) throw std::invalid_argument("zero denominator");  
    }  
  
    // Arithmetic on values returns NEW values, never mutates  
    friend Rational operator+(const Rational& a, const Rational& b) {  
        return Rational{a.num_ * b.denom_ + b.num_ * a.denom_,  
                        a.denom_ * b.denom_};  
    }  
  
    // Equality is structural -- same numerator and denominator (after reduction)  
    friend bool operator==(const Rational& a, const Rational& b) {  
        return a.num_ * b.denom_ == b.num_ * a.denom_;  
    }  
};
```


2. Value Objects – Challenge

What does this print?

```
class Money {
    int cents_;
public:
    Money(int c) : cents_{c} {}
    friend bool operator==(const Money& a, const Money& b) {
        return a.cents_ == b.cents_;
    }
};

void check(const Money& m) {
    if (m == 500)           // What happens here?
        std::cout << "five dollars\n";
    else
        std::cout << "no match\n";
}

int main() { check(Money{500}); }
```

2. Value Objects – Challenge (Solution)

What does this print?

```
class Money {
    int cents_;
public:
    Money(int c) : cents_{c} {}
    friend bool operator==(const Money& a, const Money& b) {
        return a.cents_ == b.cents_;
    }
};

void check(const Money& m) {
    if (m == 500)           // What happens here?
        std::cout << "five dollars\n";
    else
        std::cout << "no match\n";
}

int main() { check(Money{500}); }
```

Answer: It prints `five dollars`.

The constructor is NOT marked `explicit`, so the literal `500` undergoes an implicit conversion to `Money{500}` to make the `==` comparison work.

This silent conversion surprises maintainers => `m == 500` looks like a type error but compiles.

Fix: Mark single-argument constructors `explicit` to block this.

3. Mutable vs. Immutable Objects – Things to Remember

- **Entities are mutable. Values are usually immutable.** When values need to be changed (like a *list* inside it), **give access to a copy, not the original.**
- **Return copies, not references**, from getters when the member is itself mutable (e.g., `std::vector`)
- **Immutability means no public mutators** — and no member functions that hand out non-`const` references to internals
- **Mark all member functions `const`** and the class `final` to communicate the contract
- **Deleting copy/move on immutable types** makes "true immutability" enforceable, but pins objects to their initial scope
- `const` data members suppress assignment generation — the compiler cannot assign through a `const` field

3. Mutable vs. Immutable Objects – Challenge

Will this compile? If so, what does the second print show?

```
class Snapshot {
    std::vector<int> data_;
public:
    Snapshot(std::vector<int> d) : data_{std::move(d)} {}
    const std::vector<int>& view() const { return data_; } // looks safe?
};

void print(const Snapshot& s) {
    for (int x : s.view()) std::cout << x << ' ';
    std::cout << '\n';
}

int main() {
    Snapshot s{{1, 2, 3}};
    print(s);
    const_cast<std::vector<int>&>(s.view()).push_back(99); // What happens?
    print(s);
}
```

3. Mutable vs. Immutable Objects – Challenge (Solution)

Will this compile? If so, what does the second print show?

```
class Snapshot {
    std::vector<int> data_;
public:
    Snapshot(std::vector<int> d) : data_{std::move(d)} {}
    const std::vector<int>& view() const { return data_; } // looks safe?
};

void print(const Snapshot& s) {
    for (int x : s.view()) std::cout << x << ' ';
    std::cout << '\n';
}

int main() {
    Snapshot s{{1, 2, 3}};
    print(s);
    const_cast<std::vector<int>&>(s.view()).push_back(99); // What happens?
    print(s);
}
```

Answer: It compiles and prints 1 2 3 then 1 2 3 99.

The `const_cast` strips constness from the returned reference; because `s` was declared NON-const, modifying through the cast is well-defined (not UB).

Returning even a `const&` to internal mutable state means immutability is enforced by CONVENTION only => return a copy to make immutability airtight.

4. Singleton Design Pattern – Things to Remember

- **Make the constructor `private`** and delete all four copy/move special members — otherwise clients can smuggle out duplicates
- **The static instance must be defined outside the class** (e.g., `Egg Egg::instance_{42};`) or it will not link
- **Return a pointer or reference to the instance**, never a copy — `Egg::instance()` should hand out the singleton itself
- **Static initialization order across translation units is undefined** — singletons depending on other singletons need lazy init
- **`final` the class** so subclassing cannot resurrect a second instance via inheritance

4. Singleton Design Pattern – Challenge

What is the value of `Egg::instance()->val()` printed at the end?

```
class Egg {
    static Egg instance_;
    int v_;
    Egg(int v) : v_{v} { std::cout << "Egg ctor " << v << '\n'; }
public:
    Egg(const Egg&) = delete;
    static Egg* instance() { return &instance_; }
    int val() const { return v_; }
    void setVal(int v) { v_ = v; }
};

// file A.cpp
Egg Egg::instance_{42};

// file B.cpp (different TU)
struct Initializer {
    Initializer() { Egg::instance()->setVal(100); }
};
static Initializer init;

int main() {
    std::cout << Egg::instance()->val() << '\n'; // What prints?
}
```

4. Singleton Design Pattern – Challenge (Solution)

What is the value of `Egg::instance()->val()` printed at the end?

```
class Egg {
    static Egg instance_;
    int v_;
    Egg(int v) : v_{v} { std::cout << "Egg ctor " << v << '\n'; }
public:
    Egg(const Egg&) = delete;
    static Egg* instance() { return &instance_; }
    int val() const { return v_; }
    void setVal(int v) { v_ = v; }
};

// file A.cpp
Egg Egg::instance_{42};

// file B.cpp (different TU)
struct Initializer {
    Initializer() { Egg::instance()->setVal(100); }
};
static Initializer init;

int main() {
    std::cout << Egg::instance()->val() << '\n'; // What prints?
}
```

Answer: This is known as the **Static Initialization Order Fiasco (SIOF)** => the answer is "depends on link order, unspecified by the standard."

If `B.cpp`'s `init` runs BEFORE `A.cpp`'s `instance_` is constructed, `setVal(100)` operates on a not-yet-constructed object (UB), and then the constructor overwrites it back to `42`.

Classic fix: the Meyers Singleton (static local inside `instance()`) guarantees lazy, thread-safe construction.

4. Singleton Design Pattern – Challenge (Solution)

What is the value of `Egg::instance()->val()` printed at the end?

```
class Egg {
    // static Egg instance_;
    int v_;
    Egg(int v) : v_{v} { std::cout << "Egg ctor " << v << "\n"; }
public:
    Egg(const Egg&) = delete;
    // static Egg* instance() { return &instance_; }
    static Egg* instance() {
        static Egg instance_(42)
        return &instance_;
    }
    int val() const { return v_; }
    void setVal(int v) { v_ = v; }
};

// file A.cpp
Egg Egg::instance_{42}; // no need to initialize here anymore

// file B.cpp (different TU)
struct Initializer {
    Initializer() { Egg::instance()->setVal(100); }
};
static Initializer init;

int main() {
    std::cout << Egg::instance()->val() << "\n"; // What prints?
}
```

Answer: This is known as the **Static Initialization Order Fiasco (SIOF)** => the answer is "depends on link order, unspecified by the standard."

If `B.cpp`'s `init` runs BEFORE `A.cpp`'s `instance_` is constructed, `setVal(100)` operates on a not-yet-constructed object (UB), and then the constructor overwrites it back to `42`.

Classic fix: the Meyers Singleton (static local inside `instance()`) guarantees lazy, thread-safe construction.

5. PImpl Idiom (Hidden Implementation) – Things to Remember

- **Declare the destructor in the header, define it in the .cpp** — `unique_ptr<Impl>` needs `Impl` complete to destroy it
- **Use `= default`** for the destructor in the .cpp where `Impl` is complete, not in the header
- **Move operations must also be defined out-of-line** for the same reason as the destructor
- **The header holds only `struct Impl;`** forward declaration — no fields, no methods of `Impl` leak out
- **Copy constructor must `make_unique<Impl>(*other.pImpl_)`** — a defaulted copy would copy the pointer, sharing state



5. PImpl Idiom (Hidden Implementation) – Example

```
// rational.h -- what clients see
class Rational {
    struct Impl;           // incomplete type
    std::unique_ptr<Impl> pImpl_;
public:
    explicit Rational(int num = 0, int denom = 1);
    ~Rational();           // declared here, defined in .cpp
    Rational(const Rational&); // out-of-line: needs complete Impl
    Rational& operator=(const Rational&);
    Rational(Rational&&) noexcept;
    Rational& operator=(Rational&&) noexcept;
};

// rational.cpp
struct Rational::Impl { int num_, denom_; };

Rational::~~Rational() = default; // Impl is complete here
Rational::Rational(Rational&&) noexcept = default;
Rational& Rational::operator=(Rational&&) noexcept = default;

Rational::Rational(const Rational& o)
    : pImpl_{std::make_unique<Impl>(*o.pImpl_)} {} // deep copy, not pointer copy
```

5. PImpl Idiom (Hidden Implementation) – Challenge

Will this header-only pImpl class compile when a client writes `Widget w;`?

```
// widget.h
class Widget {
    struct Impl;
    std::unique_ptr<Impl> pImpl_;
public:
    Widget();
    // No destructor declared -- let the compiler default it
};

// widget.cpp
struct Widget::Impl { int data_; };
Widget::Widget() : pImpl_{std::make_unique<Impl>()} {}

// main.cpp
#include "widget.h"
int main() { Widget w; }      // What happens?
```

5. PImpl Idiom (Hidden Implementation) – Challenge (Solution)

Will this header-only pImpl class compile when a client writes `Widget w;`?

```
// widget.h
class Widget {
    struct Impl;
    std::unique_ptr<Impl> pImpl_;
public:
    Widget();
    // No destructor declared -- let the compiler default it
};

// widget.cpp
struct Widget::Impl { int data_; };
Widget::Widget() : pImpl_(std::make_unique<Impl>()) {}

// main.cpp
#include "widget.h"
int main() { Widget w; }      // What happens?
```

Answer: It FAILS to compile in `main.cpp`.

Because the destructor is not declared in the header, the compiler tries to IMPLICITLY generate it at the point of `Widget w;`'s scope exit.

At that point `Impl` is still incomplete (only forward-declared).

`unique_ptr<Impl>::~~unique_ptr()` requires `Impl` to be a complete type.

Fix: declare `~Widget();` in the header and `= default` it in the `.cpp`.

6. Default Constructor – Things to Remember

- **Built-in types are left uninitialized** by the compiler-generated default constructor — reading them is undefined behavior
- **The compiler skips generating a default constructor** the moment you declare ANY constructor, even a parameterized one
- **Use member initializers in the class body** (`int x_ = 0;`) to give built-in types a defined value with zero effort
- `= default` reinstates the default constructor after you have declared other constructors that suppressed it
- **Always use the initializer list, not the body**, to construct members — body assignment runs after default construction

6. Default Constructor – Challenge

What is the value of `m.p.x` at the `assert`?

```
struct Point {  
    int x, y;  
};  
  
struct Marker {  
    Point p;  
    int x;  
    Marker(int v) : x{v} {}    // What about p?  
};  
  
int main() {  
    Marker m{5};  
    assert(m.p.x == 0);        // Does this hold?  
}
```

6. Default Constructor – Challenge (Solution)

What is the value of `m.p.x` at the `assert`?

```
struct Point {  
    int x, y;  
};  
  
struct Marker {  
    Point p;  
    int x;  
    Marker(int v) : x{v} {}    // What about p?  
};  
  
int main() {  
    Marker m{5};  
    assert(m.p.x == 0);        // Does this hold?  
}
```

Answer: The assertion fires with undefined behavior => `m.p.x` is NOT guaranteed to be `0`.

`Point` has no user-declared constructor, but the compiler-generated default constructor leaves built-in members UNINITIALIZED.

Reading `m.p.x` reads an indeterminate value (UB).

Classic trap: programmers assume `Point p`; "zero-initializes" because `int p{};` does, but that only happens with explicit value-initialization syntax.

7. Destructor – Things to Remember

- **Raw pointer members are not `deleted`** by the compiler-generated destructor — only the pointer slot itself is reclaimed
- Use `std::unique_ptr` or `std::shared_ptr` instead of raw pointers to make resource cleanup automatic
- **Mark destructors `virtual`** in polymorphic base classes — deleting a derived object via a base pointer is UB otherwise
- **Destructors should not throw** — throwing during stack unwinding terminates the program via `std::terminate`
- **Member destructors run in reverse declaration order**, then base class destructors run last

7. Destructor – Challenge

How many C destructors run by the time main returns?

```
class C {  
public:  
    C() { std::cout << "C ctor\n"; }  
    ~C() { std::cout << "C dtor\n"; }  
};  
  
class Holder {  
    C* p_;  
public:  
    Holder() : p_{new C()} {}  
    // No destructor defined -- let the compiler default it  
};  
  
int main() {  
    { Holder h; }           // What prints?  
}
```

7. Destructor – Challenge (Solution)

How many C destructors run by the time main returns?

```
class C {
public:
    C() { std::cout << "C ctor\n"; }
    ~C() { std::cout << "C dtor\n"; }
};

class Holder {
    C* p_;
public:
    Holder() : p_{new C()} {}
    // No destructor defined -- let the compiler default it
};

int main() {
    { Holder h; }           // What prints?
}
```

Answer: Exactly ONE C ctor and ZERO C dtor outputs => the program leaks.

The compiler-generated `~Holder()` destroys the raw pointer member `p_` (reclaims the 8-byte pointer slot) but does NOT call `delete p_`.

Memory and the C instance are both leaked.

Fix: Replacing `C* p_` with `std::unique_ptr<C> p_` makes the defaulted destructor correct.

8. Copy Constructor – Things to Remember

- **The compiler default does a memberwise SHALLOW copy** — pointer members alias the source's heap data, causing double-free
- **Use the initializer list** to copy members; assignment in the body runs AFTER default construction (wasted work)
- **Copy the base subobject explicitly:** `Base(other)` in the initializer list, otherwise `Base()` runs
- **For `unique_ptr` members**, deep-copy with `make_unique<C>(*other.ptr_)` — `unique_ptr` itself is move-only
- **Take the parameter by `const MyClass&`**, never by value — by-value would require copying to call the copy constructor

8. Copy Constructor – Challenge

What does this print?

```
struct Node {  
    int* data_;  
    Node(int v) : data_{new int{v}} {}  
    ~Node() { delete data_; }  
    // Compiler-generated copy ctor -- defaults are fine, right?  
};  
  
int main() {  
    Node a{42};  
    {  
        Node b = a;           // What happens at scope exit?  
    }  
    std::cout << *a.data_ << '\n'; // What prints?  
}
```

8. Copy Constructor – Challenge (Solution)

What does this print?

```
struct Node {  
    int* data_;  
    Node(int v) : data_{new int{v}} {}  
    ~Node() { delete data_; }  
    // Compiler-generated copy ctor -- defaults are fine, right?  
};  
  
int main() {  
    Node a{42};  
    {  
        Node b = a;           // What happens at scope exit?  
    }  
    std::cout << *a.data_ << '\n'; // What prints?  
}
```

Answer: Undefined behavior (UB) => most likely a crash, a garbage value, or 42.

The compiler-generated copy constructor performs a SHALLOW copy: `b.data_` points to the SAME heap `int` as `a.data_`.

When `b` goes out of scope, `~Node()` deletes that `int`.

When `main` reads `*a.data_`, it reads freed memory, then `~a` double-frees.

The Rule of Three demands a user-defined copy constructor whenever you write a non-trivial destructor.

9. Copy Assignment Operator – Things to Remember

- **Check for self-assignment first** with `if (this == &other) return *this;` to avoid freeing then reading the same resource
- **Return `MyClass&`**, not `void` or `MyClass` — chaining (`a = b = c;`) and STL containers require this signature
- **Call `Base::operator=(other)`** explicitly if you inherit; otherwise the base subobject is not assigned
- **Free old resources before allocating new ones**, or use copy-and-swap to make the operator exception-safe
- **The parameter must be `const MyClass&`**, not `MyClass` — taking by value would copy the source first



9. Copy Assignment Operator – Challenge

What does this print?

```
struct Buffer {
    int* data_;
    Buffer(int v) : data_{new int{v}} {}
    Buffer(const Buffer& o) : data_{new int{*o.data_}} {}
    ~Buffer() { delete data_; }

    Buffer& operator=(const Buffer& o) {
        delete data_;
        data_ = new int{*o.data_};    // What if &o == this?
        return *this;
    }
};

int main() {
    Buffer a{10};
    a = a;                            // What happens?
    std::cout << *a.data_ << '\n';
}
```


9. Copy Assignment Operator – Challenge (Solution)

What does this print?

```
struct Buffer {
    int* data_;
    Buffer(int v) : data_{new int{v}} {}
    Buffer(const Buffer& o) : data_{new int{*o.data_}} {}
    ~Buffer() { delete data_; }

    Buffer& operator=(const Buffer& o) {
        delete data_;
        data_ = new int{*o.data_};    // What if &o == this?
        return *this;
    }
};

int main() {
    Buffer a{10};
    a = a;                          // What happens?
    std::cout << *a.data_ << '\n';
}
```

Answer: Undefined behavior (UB). Output may be 10 (if freed memory still holds old bytes), garbage, or a crash.

The assignment `a = a` enters with `&o == this`.

The line `delete data_` deletes `a`'s heap `int`, then `*o.data_` READS FROM THE SAME FREED POINTER before allocating the new one.

Fix: Add `if (this == &o) return *this;` at the top, or use copy-and-swap which is naturally self-assignment safe.

9. Copy Assignment Operator – Challenge (Solution)

```
struct Buffer {
    int* data_;
    Buffer(int v) : data_{new int{v}} {}
    Buffer(const Buffer& o) : data_{new int{*o.data_}} {}
    ~Buffer() { delete data_; }

    Buffer& operator=(const Buffer& o) {
        if (this == &o) { return *this; } // add here
        delete data_;
        data_ = new int{*o.data_}; // What if &o == this?
        return *this;
    }
};

int main() {
    Buffer a{10};
    a = a; // What happens?
    std::cout << *a.data_ << '\n';
}
```

Answer: Undefined behavior (UB). Output may be 10 (if freed memory still holds old bytes), garbage, or a crash.

The assignment `a = a` enters with `&o == this`.

The line `delete data_` deletes `a`'s heap `int`, then `*o.data_` READS FROM THE SAME FREED POINTER before allocating the new one.

Fix: Add `if (this == &o) return *this;` at the top, or use copy-and-swap which is naturally self-assignment safe.

10. Copy-Swap Idiom – Things to Remember

- **Exception safety is free** — if the copy constructor throws, the original object is untouched
- **Self-assignment is handled automatically** without an `if (this == &other)` guard
- **The cost is one extra allocation** even when assignment could have reused existing storage
- **Implement `swap`** as a member or friend and make it `noexcept` — STL algorithms select it via ADL
- **The idiom requires a working copy constructor and destructor** — it reuses both rather than duplicating logic



10. Copy-Swap Idiom – Example

```
class Buffer {
    int* data_;
public:
    Buffer(int v) : data_{new int{v}} {}
    Buffer(const Buffer& o) : data_{new int{*o.data_}} {}
    ~Buffer() { delete data_; }

    friend void swap(Buffer& a, Buffer& b) noexcept {
        using std::swap;
        swap(a.data_, b.data_);           // swap raw pointer slots only
    }

    // Pass by value -- triggers copy ctor; if it throws, *this is untouched
    Buffer& operator=(Buffer other) noexcept {
        swap(*this, other);                // steal copy's contents
        return *this;                      // 'other' destroys old data_
    }
};
```

10. Copy-Swap Idiom – Challenge

Does this `operator=` correctly implement copy-and-swap?

```
class Widget {  
    int* data_;  
public:  
    Widget(int v) : data_{new int{v}} {}  
    Widget(const Widget& o) : data_{new int{*o.data_}} {}  
    ~Widget() { delete data_; }  
  
    Widget& operator=(const Widget& other) { // note: const reference  
        Widget tmp{other};  
        std::swap(data_, tmp.data_);  
        return *this;  
    } // What is wrong here?  
};
```

10. Copy-Swap Idiom – Challenge (Solution)

Does this `operator=` correctly implement copy-and-swap?

```
class Widget {  
    int* data_;  
public:  
    Widget(int v) : data_{new int{v}} {}  
    Widget(const Widget& o) : data_{new int{*o.data_}} {}  
    ~Widget() { delete data_; }  
  
    Widget& operator=(const Widget& other) {    // note: const reference  
        Widget tmp{other};  
        std::swap(data_, tmp.data_);  
        return *this;  
    }                                     // What is wrong here?  
};
```

Answer: It works correctly, but MISSES the main benefit.

By taking `const Widget&` and constructing `tmp` inside, you still get exception safety and self-assignment safety.

However, the canonical version takes the parameter BY VALUE: `Widget& operator=(Widget other)`.

The by-value form lets the compiler elide the copy when the argument is an rvalue (move construction), giving you move assignment FOR FREE without writing a separate `operator=(Widget&&)`.

Pass-by-const-ref forces a copy even from rvalues.

10. Copy-Swap Idiom – Challenge (Solution)

Does this `operator=` correctly implement copy-and-swap?

```
class Widget {  
    int* data_;  
public:  
    Widget(int v) : data_{new int{v}} {}  
    Widget(const Widget& o) : data_{new int{*o.data_}} {}  
    ~Widget() { delete data_; }  
  
    Widget& operator=(Widget other) { // non-const  
        std::swap(data_, other.data_); // no tmp anymore  
        return *this;  
    }  
};
```

Answer: It works correctly, but MISSES the main benefit.

By taking `const Widget&` and constructing `tmp` inside, you still get exception safety and self-assignment safety.

However, the canonical version takes the parameter BY VALUE: `Widget& operator=(Widget other)`.

The by-value form lets the compiler elide the copy when the argument is an rvalue (move construction), giving you move assignment FOR FREE without writing a separate `operator=(Widget&&)`.

Pass-by-const-ref forces a copy even from rvalues.

11. lvalue vs. rvalue – Things to Remember

- An lvalue has an address you can take with `&` — named variables, dereferenced pointers, and subscripted arrays qualify
- An rvalue reference parameter is itself an lvalue inside the function — you must `std::move` it again to forward
- `std::move` is just a cast to `T&&` — it does not move by itself; the actual move happens in a ctor/operator
- A `const T&` can bind to rvalues, but you cannot steal from a `const` source — `move` on `const` silently copies
- Returning a local by value triggers RVO, but `return std::move(local)` DISABLES RVO and forces a real move

11. lvalue vs. rvalue – Challenge

Which constructor of Widget runs on the marked line?

```
struct Widget {  
    Widget()          { std::cout << "default\n"; }  
    Widget(const Widget&) { std::cout << "copy\n"; }  
    Widget(Widget&&) noexcept { std::cout << "move\n"; }  
};  
  
void process(Widget&& w) {  
    Widget local = w;      // What prints?  
}  
  
int main() {  
    process(Widget{});  
}
```

11. lvalue vs. rvalue – Challenge

Which constructor of Widget runs on the marked line?

```
struct Widget {  
    Widget()          { std::cout << "default\n"; }  
    Widget(const Widget&) { std::cout << "copy\n"; }  
    Widget(Widget&& noexcept { std::cout << "move\n"; }  
};  
  
void process(Widget&& w) {  
    Widget local = w;      // What prints?  
}  
  
int main() {  
    process(Widget{});  
}
```

Answer: It prints **default** then **copy**.

Inside **process**, the parameter **w** has TYPE **Widget&&**, but as a NAMED VARIABLE, **w** is an LVALUE => its name has an address.

Initializing **local** from an lvalue selects the copy constructor, not the move.

To actually move, write **Widget local = std::move(w);**

This is the single most common rvalue-reference trap in C++.

12. Move Constructor – Things to Remember

- **Mark it `noexcept`** — `std::vector` and other containers fall back to COPYING during reallocation if move can throw
- **Use `std::move`** on each member in the initializer list, otherwise the copy constructors run instead of move
- **Use `std::exchange(other.x_, 0)`** for primitives you want to "steal and reset" in one expression
- **The moved-from object must be valid for destruction and reassignment** — it does not need to be "empty," just safe
- **Move construction is SUPPRESSED** if you declare a copy constructor, copy assignment, destructor, or move assignment

12. Move Constructor – Challenge

Why is the move constructor never called?

```
class Big {
    std::vector<int> data_;
public:
    Big() : data_(1000) {}
    Big(const Big& o) : data_{o.data_} { std::cout << "copy\n"; }
    ~Big() {} // user-declared destructor
};

int main() {
    std::vector<Big> v;
    v.reserve(2);
    v.push_back(Big{});
    v.push_back(Big{}); // What prints?
}
```



12. Move Constructor – Challenge (Solution)

Why is the move constructor never called?

```
class Big {
    std::vector<int> data_;
public:
    Big() : data_(1000) {}
    Big(const Big& o) : data_{o.data_} { std::cout << "copy\n"; }
    ~Big() {} // user-declared destructor
};

int main() {
    std::vector<Big> v;
    v.reserve(2);
    v.push_back(Big{});
    v.push_back(Big{}); // What prints?
}
```

Answer: copy prints on both push_back calls.

The user-declared destructor SUPPRESSES the implicit generation of the move constructor (per the C++17 special-members table).

Even though no explicit copy/move is requested, the only available constructor for the rvalue `Big{}` is the copy constructor. To restore move generation, you must remove or `= default` both the destructor and the copy constructor, or explicitly declare `Big(Big&&) noexcept = default;`

12. Move Constructor – Challenge (Solution)

Why is the move constructor never called?

```
class Big {
    std::vector<int> data_;
public:
    Big() : data_(1000) {}
    Big(const Big& o) : data_{o.data_} { std::cout << "copy\n"; }
    ~Big() {} // user-declared destructor
    Big(Big&&) noexcept = default; // declare move constructor
};

int main() {
    std::vector<Big> v;
    v.reserve(2);
    v.push_back(Big{});
    v.push_back(Big{}); // What prints?
}
```

Answer: copy prints on both push_back calls.

The user-declared destructor SUPPRESSES the implicit generation of the move constructor (per the C++17 special-members table).

Even though no explicit copy/move is requested, the only available constructor for the rvalue `Big{}` is the copy constructor. To restore move generation, you must remove or `= default` both the destructor and the copy constructor, or explicitly declare `Big(Big&&) noexcept = default;`

13. Move Assignment – Things to Remember

- Always include the self-move guard `if (this == &other) return *this;` $\Rightarrow$ `a = std::move(a)` is rare but legal
- `noexcept` is even more critical here than for the move constructor $\Rightarrow$ exception safety of STL depends on it
- For `unique_ptr` members, plain `ptr_ = std::move(o.ptr_)` works $\Rightarrow$ it deletes the old payload and transfers
- Use `std::exchange` for built-in members to assign and zero out the source in one expression
- Return `MyClass&` for chaining and STL container compatibility, just like copy assignment



13. Move Assignment – Challenge

What is the value of `*a.data_` at the end?

```
class V {
public:
    int* data_;
    V(int x) : data_{new int{x}} {}
    V(V&& o) noexcept : data_{o.data_} { o.data_ = nullptr; }
    V& operator=(V&& o) noexcept {
        data_ = o.data_;      // What about the old data_?
        o.data_ = nullptr;
        return *this;
    }
    ~V() { delete data_; }
};

int main() {
    V a{1};
    V b{2};
    a = std::move(b);        // What happens to a's original 1?
    std::cout << *a.data_ << '\n';
}
```



13. Move Assignment – Challenge (Solution)

What is the value of `*a.data_` at the end?

```
class V {
public:
    int* data_;
    V(int x) : data_{new int{x}} {}
    V(V&& o) noexcept : data_{o.data_} { o.data_ = nullptr; }
    V& operator=(V&& o) noexcept {
        data_ = o.data_;           // What about the old data_?
        o.data_ = nullptr;
        return *this;
    }
    ~V() { delete data_; }
};

int main() {
    V a{1};
    V b{2};
    a = std::move(b);              // What happens to a's original 1?
    std::cout << *a.data_ << '\n';
}
```

Answer: It prints `2`, but `a`'s original `new int{1}` is LEAKED.

The move assignment overwrites `data_` without first deleting the existing resource.

The correct body is

```
delete data_; data_ = o.data_; o.data_ = nullptr;
```

A self-move guard is also missing.

`a = std::move(a)` would delete its own pointer then read it back.

Or use `std::swap(data_, o.data_)`, letting the moved-from destructor handle cleanup.

13. Move Assignment – Challenge (Solution)

What is the value of `*a.data_` at the end?

```
class V {
public:
    int* data_;
    V(int x) : data_{new int{x}} {}
    V(V&& o) noexcept : data_{o.data_} { o.data_ = nullptr; }
    V& operator=(V&& o) noexcept {
        if (this == &o) { return *this; } // self-move guard
        delete data_; // clean up our current existing resource FIRST
        data_ = o.data_;
        o.data_ = nullptr;
        return *this;
    }
    ~V() { delete data_; }
};

int main() {
    V a{1};
    V b{2};
    a = std::move(b);
    std::cout << *a.data_ << '\n';
}
```

Answer: It prints `2`, but `a`'s original `new int{1}` is LEAKED.

The move assignment overwrites `data_` without first deleting the existing resource.

The correct body is

```
delete data_; data_ = o.data_; o.data_ = nullptr;
```

A self-move guard is also missing.

`a = std::move(a)` would delete its own pointer then read it back.

Or use `std::swap(data_, o.data_)`, letting the moved-from destructor handle cleanup.

13. Move Assignment – Challenge (Solution 2)

What is the value of `*a.data_` at the end?

```
class V{
public:
    int* data_;
    V(int x) : data_{new int{x}} {}
    V(V&& o) noexcept : data_{o.data_} { o.data_ = nullptr; }
    V& operator=(V&& o) noexcept {
        std::swap(data_, o.data_); // swap old data with new data
        return *this;
    }
    ~V() { delete data_; }
};

int main() {
    V a{1};
    V b{2};
    a = std::move(b);
    std::cout << *a.data_ << '\n';
}
```

Answer: It prints `2`, but `a`'s original `new int{1}` is LEAKED.

The move assignment overwrites `data_` without first deleting the existing resource.

The correct body is

```
delete data_; data_ = o.data_; o.data_ = nullptr;
```

A self-move guard is also missing.

`a = std::move(a)` would delete its own pointer then read it back.

Or use `std::swap(data_, o.data_)`, letting the moved-from destructor handle cleanup.

14. Copy/Move Elision – Things to Remember

- C++17 MANDATES elision for prvalues — `T v = makeT();` constructs `v` in place with zero copy/move calls
- Named Return Value Optimization (NRVO) is permitted but not required when returning a named local variable
- Writing `return std::move(local);` PREVENTS NRVO — the compiler sees an rvalue reference, not an eligible local
- Elision can happen even when copy/move constructors have side effects — relying on those side effects is non-portable
- = delete-ing the copy constructor still allows elision in C++17 for prvalue initialization, but not for NRVO

14. Copy/Move Elision – Challenge

Which constructors run when v is initialized?

```
struct Vec {  
    Vec()          { std::cout << "default\n"; }  
    Vec(const Vec&) { std::cout << "copy\n"; }  
    Vec(Vec&&) noexcept { std::cout << "move\n"; }  
};  
  
Vec makeVec() {  
    Vec local;  
    return std::move(local);    // "Helping" the move along  
}  
  
int main() {  
    Vec v = makeVec();          // What prints?  
}
```

14. Copy/Move Elision – Challenge (Solution)

Which constructors run when v is initialized?

```
struct Vec {  
    Vec()          { std::cout << "default\n"; }  
    Vec(const Vec&) { std::cout << "copy\n"; }  
    Vec(Vec&&) noexcept { std::cout << "move\n"; }  
};  
  
Vec makeVec() {  
    Vec local;  
    return std::move(local);    // "Helping" the move along  
}  
  
int main() {  
    Vec v = makeVec();          // What prints?  
}
```

Answer: It prints `default` then `move`.

The `std::move(local)` DISABLES NRVO: instead of constructing `local` directly in the caller's `v`, the compiler treats the `return` expression as an rvalue reference and performs an actual move.

Without `std::move`, NRVO elides both the move and the temporary, printing only `default`.

Lesson: return named locals directly => `return local;` => and let the compiler do its job.

15. C++17 Special Members Table (Rule of Five) – Things to Remember

- Declaring a destructor blocks the implicit move constructor and move assignment — they become "not declared," falling back to copy
- Declaring a move operation deletes both copy operations — clients get hard compile errors on copy attempts
- `= default` lets you reinstate suppressed defaults while keeping the others user-declared
- The Rule of Five says "if one, then all" — touching any of the six special members usually means touching all six
- The Rule of Zero is even better — design members so the compiler defaults work (use `unique_ptr`, `vector`, etc.)

C++17 Special Members						
Compiler Implicitly Declares						
User Declares	Default Constructor	Destructor	Copy Constructor	Copy Assignment	Move Constructor	Move Assignment
Nothing	defaulted	defaulted	defaulted	defaulted	defaulted	defaulted
Any Constructor	not declared	defaulted	defaulted	defaulted	defaulted	defaulted
Default Constructor	user declared	defaulted	defaulted	defaulted	defaulted	defaulted
Destructor	defaulted	user declared	defaulted	defaulted	not declared	not declared
Copy Constructor	not declared	defaulted	user declared	defaulted	not declared	not declared
Copy Assignment	defaulted	defaulted	defaulted	user declared	not declared	not declared
Move Constructor	not declared	defaulted	deleted	deleted	user declared	not declared
Move Assignment	defaulted	defaulted	deleted	deleted	not declared	user declared

Defaulted User Declared Not Declared Deleted

CS 247 - Spring 2026

15. C++17 Special Members Table (Rule of Five) – Challenge

Does this compile, and which constructor runs at the marked line?

```
class Resource {  
public:  
    Resource() = default;  
    Resource(Resource&&) noexcept = default; // only move declared  
};  
  
int main() {  
    Resource r1;  
    Resource r2 = r1;           // What happens?  
}
```


15. C++17 Special Members Table (Rule of Five) – Challenge (Solution)

Does this compile, and which constructor runs at the marked line?

```
class Resource {  
public:  
    Resource() = default;  
    Resource(Resource&&) noexcept = default; // only move declared  
};  
  
int main() {  
    Resource r1;  
    Resource r2 = r1;           // What happens?  
}
```

Answer: It FAILS to compile.

Per the C++17 table, declaring a move constructor DELETES the copy constructor and copy assignment.

`Resource r2 = r1;` requires copy construction (since `r1` is an lvalue), and that operation is now defined as deleted.

The compile error is something like “use of deleted function `Resource::Resource(const Resource&)`”.

Fix: also explicitly `= default` the copy constructor if you want both.

16. Equality Operator – Things to Remember

- The compiler does not generate `operator==` in C++17 — you must write it yourself (changed in C++20 with `= default`)
- Compare pointed-to values, not pointer addresses, when your class owns heap memory through a pointer member
- Handle null cases explicitly: both null -> equal, one null -> not equal, both non-null -> dereference and compare
- Make it a `friend` or free function so both operands are converted symmetrically — left-only conversion needs non-member
- Use `[[nodiscard]]` so the compiler warns when callers ignore the result (a common bug source)

16. Equality Operator – Challenge

What does this print?

```
struct Box {  
    int* data_;  
    Box(int v) : data_{new int{v}} {}  
    Box(const Box& o) : data_{new int{*o.data_}} {}  
    ~Box() { delete data_; }  
  
    bool operator==(const Box& o) const {  
        return data_ == o.data_;           // What is this comparing?  
    }  
};  
  
int main() {  
    Box a{5};  
    Box b = a;           // deep copy via copy ctor  
    std::cout << (a == b ? "equal\n" : "different\n");  
}
```

16. Equality Operator – Challenge (Solution)

What does this print?

```
struct Box {
    int* data_;
    Box(int v) : data_{new int{v}} {}
    Box(const Box& o) : data_{new int{*o.data_}} {}
    ~Box() { delete data_; }

    bool operator==(const Box& o) const {
        return data_ == o.data_;           // What is this comparing?
    }
};

int main() {
    Box a{5};
    Box b = a;                             // deep copy via copy ctor
    std::cout << (a == b ? "equal\n" : "different\n");
}
```

Answer: It prints **different**.

The `operator==` compares `data_ == o.data_`, which compares POINTER ADDRESSES, not the integers they point to. Since the copy constructor allocates a new `int` on the heap, `a.data_` and `b.data_` are different addresses, so the result is **false**. Correct comparison is `*data_ == *o.data_` (or include null checks if pointers can be null). Most common bug when retrofitting equality onto a class with pointer members.

UNIVERSITY OF
WATERLOO



FACULTY OF MATHEMATICS
QUESTIONS?

THANK YOU!

tmellopaiva@uwaterloo.ca